

Universidade Estadual de Santa Cruz (UESC)

Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot
(Escape-Time): Implementação sequencial e paralela em memória
compartilhada com OpenMP.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Alecsander Almeida, Samuel Santos

Disciplina: DEC107 — Processamento Paralelo

Data: 22/09/2026

1. Introdução

Este relatório apresenta a primeira etapa do projeto do conjunto de Mandelbrot, esta etapa foca em implementar uma versão serial de referência e uma versão paralela utilizando OpenMP, avaliando o impacto do número de threads e das políticas de escalonamento da API.

O conjunto de Mandelbrot é definido pelo conjunto de números complexos c , tal que a recorrência $z_{n+1} = z_n^2 + c$ permaneça limitada quando iniciada em $z_0 = 0$. Na prática, calcula-se para cada ponto c do plano complexo (mapeado a partir de um pixel da imagem), o número de iterações necessárias até que o módulo de z ultrapasse o raio de escape, que neste trabalho foi definido como 2. Além disso também é necessário respeitar um número máximo de iterações (MAX_ITER), que nesse trabalho foi definido como 1000 para a imagem normal, e como 5000 para o teste de estresse do vale dos cavalos marinhos. O escape time também é utilizado em cada pixel da imagem, para definir a cor da imagem final.

O paralelismo implementado usando API OpenMP, permite que várias threads sejam executadas simultaneamente sobre o mesmo espaço de memória. A diretiva `#pragma omp parallel for`, consegue distribuir as iterações de um laço entre as threads disponíveis no computador, e a cláusula `schedule` consegue controlar como essa distribuição é feita. Nessa etapa utilizamos 3 políticas:

- `static`: o trabalho é dividido em blocos de tamanhos fixos (chamados de chunks) e atribuídos as threads de forma previsível, antes da execução começar. Quando nenhum

chunk é especificado no código, o compilador divide o laço em blocos contínuos aproximadamente iguais, um por thread.

- dynamic: o trabalho é dividido em chunks que são distribuídos dinamicamente sob demanda. Quando um thread termina seu chunk, ele solicita um novo em tempo de execução (runtime).
- guided: ele é semelhante ao dynamic, porém as chunks diminuem progressivamente o tamanho ao longo da execução, começando com chunks maiores que vão reduzindo conforme o laço chega ao fim.

O conjunto de Mandelbrot é classificado como um problema embaraçosamente paralelo, por conta do cálculo do escape time de cada pixel ser independente. No entanto, o custo computacional de cada pixel varia: pixels fora do conjunto escapam em poucas iterações, enquanto os pixels do conjunto ou próximos a ele chegam na quantidade máxima de iterações permitidas. Essa diferença de custo por pixel faz com que uma divisão de trabalho baseada na quantidade de pixels, e não no custo real de cada um resulte em threads com cargas de trabalho muito diferentes. Por isso, a análise de balanceamento de carga é o ponto central dessa etapa, como também entender como cada política de escalonamento redistribui esse desbalanceamento entre as threads, principalmente no teste do vale dos cavalos marinhos.

A análise desse desempenho em sistemas de memórias compartilhadas é importante pois o simples uso de mais threads não garante, por si só, uma redução proporcional no tempo de execução. Vários fatores afetam o ganho real obtido com o paralelismo. Assim, entender o problema, entender profundamente a API, e medir e comparar métricas entre tempo de execução, speedup, eficiência, balanceamento de carga, é essencial para verificar se o paralelismo está sendo aproveitado, ou se a aplicação está sendo limitada por gargalos, que apenas adicionar threads não resolve.

2. Metodologia

Na versão serial mapeamos as coordenadas de cada pixel da imagem (px,py), para um ponto correspondente no plano complexo, delimitado pelos parâmetros x_{min} , x_{max} , y_{min} , y_{max} . A partir de $z_0 = 0$, iteramos sobre a recursão do problema, e mantivemos as partes reais e imaginárias de z em variáveis double separadas. O critério de escape utilizamos o quadrado do módulo ($z_{real}^2 + z_{imaginary}^2 > 4$) como pedido. Armazenamos cada ponto de escape (pixel) em uma matriz em ordem row major e armazenamos como int16_t (16 bits), que atende tanto o escape de 1000, quanto o de 5000. Essa implementação serial foi a que utilizamos de referência para validar a corretude da versão OpenMP, tanto quanto a medida de tempo para o speedup.

A versão paralela reaproveita a função de cálculo do escape time da versão serial, e paraleliza o laço que percorre as linhas (py) da imagem, por meio da diretiva `#pragma omp parallel for`, combinada com a cláusula `schedule`. A paralelização foi aplicada sobre as linhas e não sobre

as colunas para não causar um grande scheduling overhead do openmp, e, além disso, porque teoricamente, pode preservar uma boa localidade de memória (deixando os pixels da mesma linha no cache do processador), deixando o programa mais rápido. Fora isso não foi necessário adicionar a clausula collapse pois a granularidade já fica suficientemente boa, mesmo dividindo em 16 threads (256 linhas/ 1 milhão de pixels por thread).

Sobre a coleta dos dados para calcular o fator de balanceamento de carga, implementamos uma variante do laço paralelo. Nessa versão, o código soma o número real de iterações que cada thread efetivamente executou (e não apenas a quantidade de linhas atribuídas a ela), armazenando esse total em uma struct dedicada para cada thread. Um detalhe importante dessa implementação é que preenchemos essa struct com dados vazios (padding) até que ela completasse 64 bytes, que é o tamanho típico de uma linha de cache do processador. Fizemos isso para evitar o efeito de false sharing (compartilhamento falso), um problema que ocorre quando threads diferentes tentam escrever simultaneamente em variáveis vizinhas de um mesmo array, forçando o hardware a invalidar constantemente a linha de cache umas das outras e destruindo o desempenho. Por fim, garantimos que essa instrumentação seja utilizada apenas quando solicitamos as métricas de balanceamento. Assim, a versão principal de medição de tempo puro se mantém livre desse custo adicional, garantindo que o cálculo de speedup não sofra interferências.

Foram avaliadas as 3 políticas de escalonamento oferecidas pelo schedule do openmp, como dito na introdução. Além disso cada política foi testada com tamanhos de chunk de 1,4,16,64 linhas, com exceção do experimento de escalabilidade forte, que utiliza static sem chunk explícito, não tendo nenhum overhead da API. A comparação entre as políticas e chunks foi realizada com o número máximo de threads disponível no ambiente de teste, tanto para o input padrão, quanto para o caso do vale dos cavalos marinhos.

Os experimentos foram realizados em uma máquina com processador AMD Ryzen 7 7735HS with Radeon Graphics, com 16 processadores lógicos detectados pelo sistema operacional, executando Windows 11 (build 10.0.26200). A compilação foi feita com o compilador Microsoft C/C++ (cl.exe), a partir de um terminal de desenvolvedor do Visual Studio configurado para x64, usando a linha:

```
cl /nologo /std:c11 /O2 /W4 /WX /openmp benchmark.c serial.c paralel.c settings.c  
/Fe:benchmark.exe
```

Essa linha utiliza otimização /O2, padrão C11, suporte a OpenMP habilitado via /openmp, com avisos tratados como erro (/W4 /WX) para garantir a ausência de comportamentos indefinidos não sinalizados pelo compilador. O programa de benchmark limita o experimento a, no máximo, 16 threads, mesmo em máquinas com mais processadores lógicos, na máquina do teste, o próprio hardware já limita o experimento a 16 threads.

O tempo de cada execução foi medido utilizando a função `omp_get_wtime()`, chamada antes e depois da função de cálculo de escape time, de forma que o intervalo de tempo que medimos corresponda ao tempo de cálculo, excluindo o tempo de por exemplo, escrita dos resultados em disco. Cada configuração testada no benchmark foi executada 3 vezes, e a mediana dessas 3 medições foi adotada como estatística principal para o cálculo de speedup e eficiência, que reduz a influência de oscilações pontuais causadas pelo sistema operacional ou o hardware. Antes de cada execução paralela, o número de threads foi fixado explicitamente com `omp_set_num_threads`, com ajuste dinâmico do número de threads desabilitado, garantido que o número de threads utilizado seja o número solicitado na diretiva.

Sobre as métricas de avaliação, como citado nos outros parágrafos, que foram calculadas a partir das medianas de tempos obtidas:

- Tempo de execução: tempo, em segundos, gasto exclusivamente no cálculo da matriz de escape time.
- Speedup: razão entre a mediana do tempo de execução da versão serial e a mediana do tempo de execução da versão paralela, para um dado número de threads: $\text{speedup}(p) = t_{\text{serial}}/t_{\text{paralelo}}(p)$, tal que p é o número de threads.
- Eficiência: razão entre o speedup obtido e o número de threads utilizadas, indicando o quão bem cada thread está sendo aproveitada: $\text{eficiencia}(p) = \text{speedup}(p)/p$.
- Fator de balanceamento de carga: razão entre a carga da thread mais sobrecarregada e a carga média de todas as threads, onde a carga de cada thread é medida pela soma das iterações realmente realizadas por ela: $\text{fator} = \text{carga_maxima}/\text{carga_media}$. Um valores próximos a 1 representam uma boa distribuição, enquanto valores muito altos indicam threads muito sobrecarregadas em relação a média.

3. Resultados

Inicialmente, foi realizado o teste de corretude, cujo objetivo foi medir a margem de erro entre os algoritmos paralelo e sequencial. Para isso, foi analisada a diferença na geração dos pixels produzidos por ambas as implementações. Por se tratar de um algoritmo determinístico, não são esperadas diferenças significativas entre os resultados obtidos. Entretanto, podem ocorrer pequenas variações decorrentes de aproximações em operações com números de ponto flutuante. Essas diferenças, quando presentes, são esperadas em nível computacional e devem permanecer dentro de uma margem de erro previamente estabelecida.

Assim, para a verificação da corretude, foi desenvolvida uma matriz de testes baseada na contagem de iterações em diferentes configurações de execução. O objetivo dessa abordagem foi reduzir a possibilidade de que a corretude observada estivesse associada apenas a um ambiente ou configuração específica. Com isso, foram avaliadas diferentes quantidades de threads, sendo utilizadas 1, 2, 4, 8 e 16 threads; diferentes políticas de escalonamento, incluindo *static*, *dynamic* e *guided*; e diferentes tamanhos de *chunks*, com valores 1, 4, 16 e 64. Também foram realizados testes de escalabilidade fraca, utilizando como referência a resolução padrão de 4096x4096. Todas essas configurações foram avaliadas tanto no caso

padrão do conjunto de Mandelbrot quanto na região conhecida como Vale dos Cavalos-Marinheiros, permitindo verificar a corretude do algoritmo em cenários com diferentes níveis de complexidade computacional.

Foram realizados 5 testes de escalabilidade forte para o caso padrão. Além disso, foram executados 24 testes envolvendo o caso padrão e a região *Seahorse Valley*, considerando as políticas de escalonamento *static*, *dynamic* e *guided*, combinadas com diferentes tamanhos de *chunks*. Dessa forma, foi possível avaliar o comportamento do algoritmo sob diferentes estratégias de distribuição da carga de trabalho e em regiões com diferentes níveis de complexidade computacional.

case	policy	chunk	threads	different_pixels	max_abs_difference	allowed_different_pixels	allowed_max_abs_difference	status
standard	static	0	1	0	0	1677	1	APPROVED_EXACT
standard	static	0	2	0	0	1677	1	APPROVED_EXACT
standard	static	0	4	0	0	1677	1	APPROVED_EXACT
standard	static	0	8	0	0	1677	1	APPROVED_EXACT
standard	static	0	16	0	0	1677	1	APPROVED_EXACT
standard	static	1	16	0	0	1677	1	APPROVED_EXACT
seahorse_valley	static	1	16	0	0	1677	1	APPROVED_EXACT
standard	static	4	16	0	0	1677	1	APPROVED_EXACT
seahorse_valley	static	4	16	0	0	1677	1	APPROVED_EXACT
standard	static	16	16	0	0	1677	1	APPROVED_EXACT
seahorse_valley	static	16	16	0	0	1677	1	APPROVED_EXACT
standard	static	64	16	0	0	1677	1	APPROVED_EXACT
seahorse_valley	static	64	16	0	0	1677	1	APPROVED_EXACT

standard	dynamic	1	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	dynamic	1	16	0	0	1677	1	APPROVE D_EXACT
standard	dynamic	4	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	dynamic	4	16	0	0	1677	1	APPROVE D_EXACT
standard	dynamic	16	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	dynamic	16	16	0	0	1677	1	APPROVE D_EXACT
standard	dynamic	64	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	dynamic	64	16	0	0	1677	1	APPROVE D_EXACT
standard	guided	1	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	guided	1	16	0	0	1677	1	APPROVE D_EXACT
standard	guided	4	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	guided	4	16	0	0	1677	1	APPROVE D_EXACT
standard	guided	16	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	guided	16	16	0	0	1677	1	APPROVE D_EXACT
standard	guided	64	16	0	0	1677	1	APPROVE D_EXACT
seahorse_ valley	guided	64	16	0	0	1677	1	APPROVE D_EXACT

Tabela 1: Matrix de Iteração para Corretude

Com isso, foi possível avaliar a corretude do algoritmo paralelo em relação ao algoritmo sequencial. Nas 29 configurações testadas, não foram observadas diferenças significativas nos valores obtidos quando comparados aos resultados do algoritmo sequencial, indicando que a implementação paralela manteve a corretude esperada nos diferentes cenários avaliados.

Também foi realizada a avaliação do caso de entrada padrão para as execuções sequencial e paralela, variando o número de threads utilizadas. Para essa análise, foram utilizadas as métricas de *Speedup* e Eficiência.

threads	serial_median_s	parallel_median_s	speedup	efficiency	efficiency_percent
1	11.903291800	8.827480800	1.348436	1.348436	134.844
2	11.903291800	3.905080000	3.048156	1.524078	152.408
4	11.903291800	3.792586600	3.138568	0.784642	78.464
8	11.903291800	2.798289800	4.253774	0.531722	53.172
16	11.903291800	1.825739700	6.519709	0.407482	40.748

Tabela 2: Tempos obtidos para o input padrão 4096x4096 e diferentes threads

Input padrão: 4096x4096, MAX_ITER = 1000

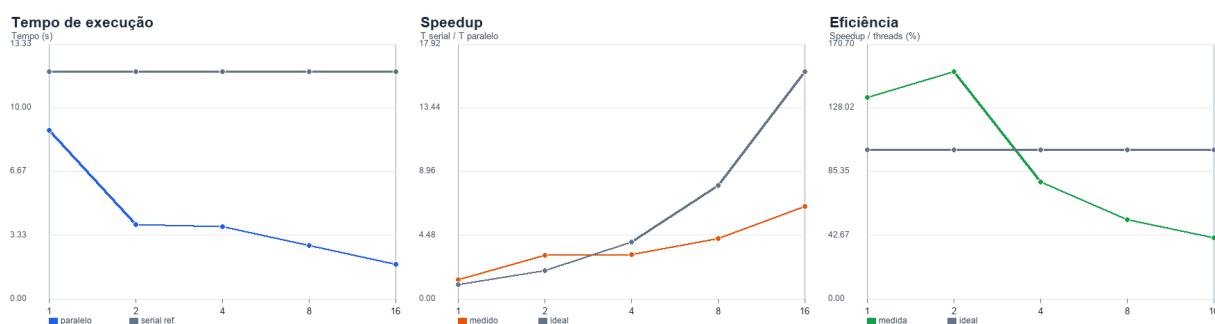


Imagem 1: Imagem para Tempo de Execução, Speedup, Eficiência para o Input Padrão

A análise do *Speedup* demonstrou uma rápida tendência de estagnação conforme o número de threads foi aumentado. A Eficiência, por sua vez, apresentou um pico nas primeiras configurações testadas, seguido por uma redução à medida que novas threads foram adicionadas. A partir desses resultados, é possível inferir que o aumento contínuo do número de threads proporciona ganhos cada vez menores de desempenho, indicando que a adição de novos recursos computacionais não necessariamente resulta em um aumento proporcional da eficiência.

Esse comportamento também pode ser analisado a partir da Lei de Amdahl, que descreve o limite teórico de *Speedup* para problemas de tamanho fixo considerando a existência de uma parcela sequencial da execução. Para o caso padrão, utilizando a política *static* e 16 threads, foi estimada uma fração sequencial de aproximadamente 9,69%, resultando em um *Speedup* máximo teórico de aproximadamente 10,32×. Dessa forma, mesmo considerando uma quantidade teoricamente infinita de threads, o tempo de execução não poderia ser reduzido abaixo de aproximadamente 1,154 s.

Com esses resultados, é possível compreender melhor a relação entre a quantidade de recursos utilizados e o desempenho obtido pelo algoritmo de Mandelbrot. A partir de determinado número de threads, o ganho adicional passa a ser limitado pela parcela sequencial do algoritmo e pelos custos associados à paralelização, tornando o aumento do número de threads cada vez menos vantajoso para um problema de tamanho fixo.

Políticas de escalonamento com 16 threads

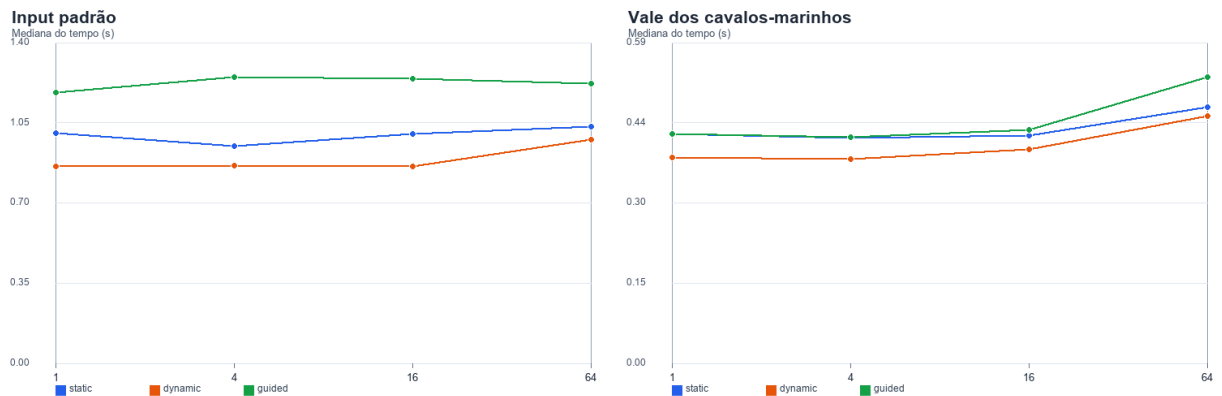


Imagem 2: Política de escalonamento com 16 threads

A ideia deste ensaio é demonstrar como o tamanho dos *chunks* influencia diretamente o balanceamento da carga de trabalho. *Chunks* maiores reduzem a quantidade de vezes em que o trabalho precisa ser distribuído entre as threads, diminuindo o custo de agendamento. Entretanto, essa abordagem pode fazer com que uma determinada thread receba um bloco de processamento significativamente mais custoso que os demais. Como o conjunto de Mandelbrot apresenta regiões com diferentes níveis de complexidade computacional, *chunks* pequenos ou médios permitem uma redistribuição mais eficiente da carga de trabalho entre as threads. O dynamic obteve o melhor resultado da política de escalonamento com o melhor aproveitamento de tempo dentre todos.

Balanceamento de carga: vale dos cavalos-marinhos

Carga máxima / carga média

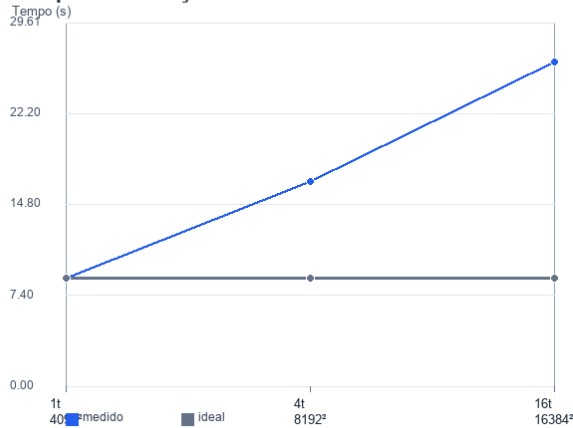


Imagem 3: Balanceamento de Carga

Na análise de balanceamento de carga, observa-se que o aumento do tamanho do *chunk* tende a elevar o desbalanceamento entre as threads. Para a política *static*, o fator de balanceamento variou de 1,002, com *chunk* 1, para 1,258, com *chunk* 64. Já na política *dynamic*, esse valor variou de 1,122 para 1,329. Dessa forma, é possível observar que *chunks* maiores podem concentrar uma parcela maior da carga de trabalho em determinadas threads, aumentando o desbalanceamento e, consequentemente, o tempo de execução.

Escalabilidade fraca: 4096² pixels por thread

Tempo de execução



Eficiência fraca

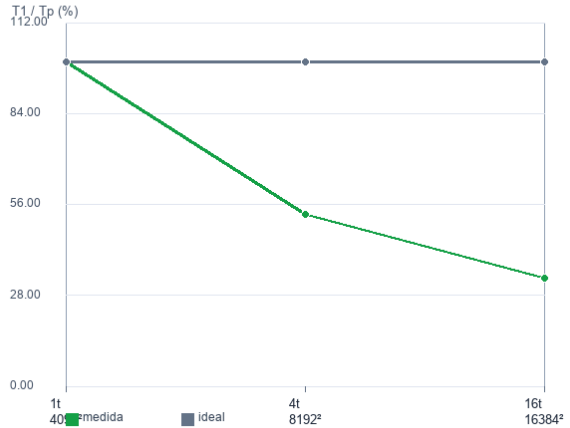


Imagem 4: Teste de escalabilidade fraca

Neste teste de escalabilidade fraca, foi avaliado o aumento da carga de trabalho em relação ao crescimento do número de threads. Com isso, foi possível observar um comportamento compatível com a Lei de Gustafson, na qual o aumento do número de unidades de processamento pode proporcionar ganhos de desempenho à medida que a carga de trabalho também cresce. No entanto, os resultados obtidos permaneceram distantes do comportamento ideal descrito pela Lei de Gustafson, indicando a presença de limitações na escalabilidade do problema analisado.

4. Discussão

O algoritmo sequencial gerou de maneira adequada os exemplos do conjunto de Mandelbrot e, com base nesses resultados, foi possível comparar a saída obtida pelo algoritmo paralelo. Ambos apresentaram resultados equivalentes, permitindo verificar a correção da implementação paralela. Além disso, os valores obtidos permaneceram dentro do limite máximo de erro estabelecido para que a solução fosse considerada correta. Portanto, a corretude do modelo paralelo mostrou-se adequada.

O algoritmo paralelo alcançou, em seu melhor resultado dentro do recorte de testes realizado, um ganho de eficiência de aproximadamente 152%, considerando a utilização de 2 threads. Esse resultado demonstra um desempenho satisfatório da abordagem paralela para a resolução do problema computacional proposto, evidenciando os benefícios da paralelização em relação à execução sequencial.

Para este problema, o aumento do número de threads apresentou um impacto significativo no ganho de *speedup*. Nos testes em que a carga de trabalho foi aumentada juntamente com a quantidade de threads, também foi observado um crescimento do *speedup*, indicando um melhor aproveitamento do paralelismo à medida que o tamanho do problema aumentava. Apesar disso, esse crescimento permaneceu abaixo do comportamento ideal, evidenciando a presença de custos adicionais e limitações na distribuição da carga de trabalho.

Por outro lado, nos testes realizados com tamanho fixo do problema, o ganho de *speedup* apresentou uma tendência de estagnação conforme o número de threads aumentava. Esse comportamento ocorre porque a quantidade de trabalho disponível permanece limitada, reduzindo a possibilidade de distribuir novas tarefas de maneira eficiente entre um número cada vez maior de threads. Dessa forma, enquanto o aumento conjunto da carga de trabalho e do número de threads favorece a escalabilidade, a manutenção de um problema de tamanho fixo impõe um limite prático ao ganho de desempenho obtido com a paralelização.

Em relação ao problema de balanceamento de carga, observou-se que o aumento da quantidade de *chunks* trouxe consigo um crescimento considerável do desbalanceamento da carga de trabalho. Esse comportamento ocorre principalmente devido à diferença na quantidade de cálculos necessários ao longo das imagens geradas, uma vez que regiões mais detalhadas tendem a exigir um número maior de operações quando comparadas a regiões mais simples. Dessa forma, foi possível observar que a distribuição do trabalho entre as threads não ocorreu de maneira uniforme, resultando em um aumento do desbalanceamento conforme a configuração dos *chunks* utilizada.

Entretanto, esse crescimento não foi linear e não permite concluir diretamente o comportamento para problemas maiores, pois o tamanho do problema permaneceu fixo. Apesar de apresentar maior desbalanceamento que *static*, a política *dynamic* obteve os menores tempos. Isso indica que o desempenho também é influenciado pela sobrecarga de agendamento e pela localização da memória. Ainda assim, todas as configurações paralelas foram mais rápidas que a execução sequencial.

5. Conclusão

Ao longo do desenvolvimento desta etapa, foi possível aprofundar os conhecimentos sobre o OpenMP e sobre diferentes formas de abstração de hardware aplicadas ao processamento paralelo. Com isso, tornou-se possível desenvolver uma abordagem paralela mais aprofundada tanto sobre o problema computacional proposto quanto sobre as técnicas utilizadas para solucioná-lo.

O principal desafio encontrado foi o desbalanceamento de carga, pois diferentes regiões da imagem exigem quantidades distintas de iterações. Dessa forma, mesmo quando as threads recebem quantidades semelhantes de linhas, algumas podem processar mais trabalho que outras e permanecer ativas por mais tempo.

Como proposta para reduzir esse problema, foi utilizada a divisão do trabalho em *chunks*, permitindo, em alguns casos, uma melhor distribuição da carga entre as threads. Entretanto, apesar da melhora observada, essa estratégia ainda não foi suficiente para solucionar completamente o problema de desbalanceamento, pois o desempenho também depende da sobrecarga de agendamento e da localidade de memória.

6. Referências

- WIKIPEDIA. Plotting algorithms for the Mandelbrot set. Disponível em: <https://en.wikipedia.org/wiki/Plotting_algorithms_for_the_Mandelbrot_set>. Acesso em: 21 set. 2026.
- UNIVERSITY OF CALIFORNIA, SAN DIEGO (UCSD). Lecture 14 (CSE 160: Spring 2005). Disponível em: <<https://cseweb.ucsd.edu/groups/csag/html/teaching/cse160s05/lectures/Lecture14.pdf>>. Acesso em: 21 set. 2026.
- QUILEZ, Inigo. Palettes. Disponível em: <<https://iquilezles.org/articles/palettes/>>. Acesso em: 21 set. 2026.
- AMDAHL, Gene M. *Validity of the single processor approach to achieving large scale computing capabilities*. Disponível em: <https://doi.org/10.1145/1465482.1465560>. Acesso em: 21 set. 2026.
- GUSTAFSON, John L. *Reevaluating Amdahl's Law*. Disponível em: <https://doi.org/10.1145/42411.42415>. Acesso em: 21 set. 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Alecsander Augusto Paixão Almeida, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Claude Sonnet 5, Chatgpt, Gemini 3.1 Pro, Gemini 3.8 Flash. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Revisão gramatical de trechos do relatório
- Auxílio na depuração (debugging) de blocos específicos de código
- Estruturação inicial de ideias e revisão bibliográfica
- Pesquisa de fontes
- Entender o problema e aprender o conteúdo relacionado a ele.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna - BA, 21/09/2026

Alecsander Augusto Paixão Almeida

Matrícula: 202311087

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, e Samuel Ferreira Santos, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "[Título do Trabalho]", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: ChatGPT 6.0

- Automação de processos de benchmarking
- Busca de fontes
- Revisão gramatical
- Debate de validade e argumentação do artigo
- Esclarecimento de pontos do trabalho e observações de lacunas

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus, 22/09/2026

Samuel Ferreira Santos

Matrícula: 202410518